MINISTRY OF EDUCATION & TRAINING

**HO CHI MINH CITY UNIVERSITY OF ECONOMICS AND FINANCE**




# FINAL REPORT

**MOBILE DEVICE PROGRAMMING**

# LIBRARY MANAGEMENT APPLICATION

## FINAL REPORT

Ho Chi Minh - <2026>

MINISTRY OF EDUCATION & TRAINING

**HO CHI MINH CITY UNIVERSITY OF ECONOMICS AND FINANCE**




# FINAL REPORT

## MOBILE DEVICE PROGRAMMING

# LIBRARY MANAGEMENT APPLICATION

**Members:**

| | | |
|---|---|---|
| **Nguyễn Hiếu Nghĩa** | **-** | **….** |
| **Phương Tiến Duy** | **-** | **235051674** |
| **Lê Minh Dũng** | **-** | **235057515** |
| **Dương Minh Chiến** | **-** | **235056681** |
| **Trịnh Trí Cường** | **-** | **….** |

Ho Chi Minh - <2026>

# Chapter 1 . INTRODUCTION

## 1.1 OVERVIEW OF THE TOPIC

In recent years, digital transformation has significantly changed the way organizations manage information and provide services to users. Libraries are no exception, as traditional library management methods based on paper records or standalone computer systems may become inefficient when the number of books and readers increases. Manual processes can make it difficult to search for books, track borrowing and returning activities, manage reservations, and maintain accurate information about book availability.

The Library Management Application is a mobile-based solution designed to digitalize and simplify the main activities of a library. The system provides a convenient platform that allows readers to search for books, view book details, reserve books, monitor their current borrowings, request borrowing extensions, and manage their personal profiles directly from a mobile device.

In addition to reader-oriented functions, the application also provides administrative features to support daily library operations. Administrators and librarians can manage books and categories, process borrowing and returning transactions, approve reservation requests, manage reader accounts, configure borrowing regulations, and monitor library statistics.

By integrating these functions into a centralized mobile application, the system aims to improve the efficiency of library operations, reduce manual work, minimize data inconsistencies, and provide readers with a more convenient way to access library services.

## 1.2 BACKGROUND AND PROBLEM STATEMENT

Traditional library management usually requires readers to visit the library directly to search for books, check book availability, borrow books, or ask librarians about their borrowing status. In some cases, information about books and borrowing transactions is recorded manually or maintained through separate management tools.

This approach can create several operational difficulties. Readers may spend significant time searching for a particular book without knowing whether the book is currently available. They may also have difficulty keeping track of borrowing dates and return deadlines. When a popular book is unavailable, there may not be a convenient mechanism for readers to reserve the book in advance.

From the library management perspective, librarians must manage a large amount of information related to books, readers, borrowing records, reservations, return status, and library regulations. Manual processing may increase the possibility of mistakes, such as incorrect borrowing records, inaccurate book quantities, forgotten return deadlines, or inconsistent reader information.

Another challenge is maintaining communication between the library and its readers. Important information such as reservation status, borrowing notifications, or general library announcements may not reach readers quickly if the system does not provide an integrated notification mechanism.

Therefore, a centralized Library Management Application is proposed to address these problems. The mobile application enables readers to access library services conveniently while also providing administrators and librarians with tools to manage books, borrowing activities, reader accounts, reservations, and library policies more efficiently.

## 1.3 OBJECTIVES

The primary objective of the **Library Management Application** is to develop a mobile platform that supports both readers and library administrators in performing common library activities efficiently and conveniently. The system is designed to digitalize traditional library processes, improve access to library services, reduce manual

administrative work, increase the accuracy of borrowing and returning records, and provide centralized management of books, readers, reservations, and library transactions.

For readers, the application aims to provide a convenient and intuitive mobile interface where users can register and manage their accounts, search for books, view detailed book information, check availability, reserve books, and monitor their current borrowing activities. Readers can also keep track of borrowing dates and return deadlines, while the renewal function allows eligible borrowed books to be extended when no other reader has placed a reservation for them.

For administrators and librarians, the application is intended to simplify daily operational tasks by providing tools for managing books, categories, storage locations, reader accounts, librarian accounts, borrowing transactions, return transactions, and reservation requests. The system also supports the configuration of fundamental library policies such as borrowing duration, maximum borrowing quantity, and penalty rates.

In addition, the system aims to provide useful statistical information that helps administrators monitor library activities, including the number of borrowed books, overdue books, popular books, and newly registered readers. Through these functions, the project is expected to improve operational efficiency, data accuracy, service quality, and the overall user experience of library management.

## 1.4 SCOPE AND LIMITATIONS

The Library Management Application is developed as a mobile-based system that supports the essential activities of a library. The system focuses on two main user groups, including Readers and Administrators, while also providing common functions that can be accessed by authenticated users. The application is designed to simplify book searching, borrowing management, reservation processing, user management, and other daily library operations. The scope of the project is defined based on the functional requirements established for the academic development environment.

### 1.4.1 Scope

The system provides common account management functions such as registration, login, logout, profile management, password changes, and system notifications. Users can update personal information including their name, phone number, and avatar through the mobile application.

For Readers, the system provides functions for searching and exploring the library's book collection. Readers can search for books using keywords related to book titles or authors and can filter search results based on categories and publication years. Each book contains detailed information such as title, author, summary, number of pages, publisher, publication year, available quantity, and physical storage location within the library.

The application also allows Readers to reserve books for future borrowing. This function is useful when a book is temporarily unavailable or when a reader wants the library to hold an available book for later collection. Readers can view their current borrowing records, including the books being borrowed, borrowing dates, and due dates. The system also supports renewal requests, allowing readers to extend the borrowing period if the selected book has not been reserved by another reader.

Depending on the final implementation, Readers may also be able to access their borrowing history, view overdue penalty information, and display an electronic library card through a QR code or barcode on their mobile device.

For Administrators and Librarians, the system provides management functions for books, categories, storage locations, readers, and staff accounts. Authorized users can add, update, and remove book information, manage book quantities, classify books into appropriate categories, and define the storage location of books according to buildings, floors, or shelves.

The application supports the borrowing and returning process by allowing librarians to identify readers and books through QR codes or barcodes when the relevant feature is implemented. During the returning process, librarians can confirm that a book has been returned and record its physical condition, such as normal, damaged, or lost.

Administrators can also review and approve book reservation requests submitted by Readers. In addition, the system provides functions for managing librarian accounts, maintaining reader information, and locking or unlocking library cards when readers violate library regulations.

The application also supports system configuration for essential library policies, including the maximum borrowing period, the maximum number of books that a reader can borrow at one time, and the daily overdue penalty rate. Basic statistical functions are provided to help administrators monitor the number of books currently being borrowed, overdue books, frequently borrowed books, and newly registered readers.

### 1.4.2 Limitations

Although the **Library Management Application** provides the essential functions required for managing daily library activities, the current version is developed within the limitations of an academic project. Therefore, the system primarily focuses on the operations of a single library environment and does not provide complete support for large-scale library networks with multiple independent branches and complex inter-library book transfer processes.

Some functions are considered optional and may not be fully implemented in the current version depending on development time and technical conditions. These functions include detailed borrowing history, penalty management, electronic library cards using QR codes or barcodes, and statistical reports related to penalty revenue.

The current application mainly manages physical books stored within the library. Digital library functions such as electronic books, online book reading, digital content licensing, and Digital Rights Management are not included in the scope of this project.

The statistical and reporting functions are also limited to basic operational information. Advanced functions such as personalized book recommendations, predictive analytics, artificial intelligence-based search, and comprehensive business intelligence dashboards are not implemented in the current version.

Furthermore, the application is designed and evaluated within a controlled academic development environment. Its performance under a very large number of concurrent users, books, and transactions has not been tested at an enterprise scale. Large-scale deployment may require additional infrastructure, database optimization, security improvements, and performance testing.

External integrations such as online payment gateways for penalty payments, national library systems, third-party book databases, or other external information systems are also outside the primary scope of the current project.

## 1.5 PROJECT SIGNIFICANCE

The development of the **Library Management Application** is significant because it provides a practical solution for improving both library operations and reader experience. By digitalizing common library activities, the system helps reduce the dependency on manual processes and allows library information to be managed in a more centralized and consistent manner.

For Readers, the application provides a convenient way to access library services through a mobile device. Users can search for books, check book availability, view detailed information, place reservations, and monitor their current borrowing status without having to directly contact library staff for every request. This improves accessibility to library resources and reduces the time required to obtain information about books.

The borrowing management functions also help Readers monitor borrowing dates and return deadlines more effectively. By providing clear information about current borrowing records and supporting renewal requests, the application can help reduce accidental overdue returns and improve compliance with library regulations.

For Administrators and Librarians, the system provides tools to manage books, reader information, reservations, borrowing transactions, and returning transactions through a centralized platform. This reduces repetitive manual work and minimizes the risk of errors caused by paper-based records or disconnected management processes.

The application also contributes to more accurate inventory management by allowing book quantities, categories, and physical storage locations to be maintained systematically. Librarians can therefore obtain updated information about the status and location of books more efficiently.

In addition, reporting and statistical functions provide administrators with useful information for evaluating library activities. Data related to borrowed books, overdue books, popular books, and newly registered readers can assist library administrators in understanding usage patterns and making better management decisions.

From an academic perspective, the project provides an opportunity to apply knowledge related to mobile application development, system requirement analysis, database design, user interface design, authentication, authorization, and software engineering. The project therefore not only provides practical value for library management but also helps the development team gain experience in designing and implementing a complete information system.

## 1.6 MEMBERS & WORK ASSIGNMENT (CHIA LẠI SAU)

| Name | ID | Mission | Contribute |
|---|---|---|---|
| Nguyễn Hiếu Nghĩa | 235057450 | | % |
| Phương Tiến Duy | 235051674 | | |
| Lê Minh Dũng | 235057515 | | |
| Dương Minh Chiến | 235056681 | | |
| Trịnh Trí Cường | | | |

# Chapter 2 . LITERATURE REVIEW & REQUIREMENTS

## 2.1 TECHNOLOGIES

The **Library Management Application** is developed as a mobile application using **Android Studio** as the primary development environment. Android Studio provides the necessary tools for designing user interfaces, writing application logic, managing project resources, testing features, and running the application on Android devices or emulators. It is suitable for the development of this project because it provides an integrated environment specifically designed for Android application development.

The application is built for the Android platform and focuses on providing a convenient mobile interface for both Readers and Administrators. Through the application, users can perform common library activities such as account authentication, book searching, book reservation, borrowing management, returning management, profile updates, and administrative operations.

For data storage, the system uses **SQLite**, a lightweight relational database that can be integrated directly into Android applications. SQLite is used to store and manage the main data of the system, including user accounts, books, categories, borrowing records, reservation requests, and other related information. Since SQLite does not require a separate database server, it is appropriate for an academic mobile application and simplifies the deployment and testing process.

The relational database structure allows the application to maintain relationships between different types of data. For example, borrowing records are associated with Readers and Books, reservation records are linked to specific users and books, and books can be classified into categories and assigned to specific storage locations. This structure helps maintain data consistency and supports efficient data retrieval during system operations.

Android Studio and SQLite together provide a practical development environment for implementing the core functions of the **Library Management Application**. Android Studio supports the design and development of the mobile interface and application logic, while SQLite provides persistent local data storage for the library management information.

## 2.2 REQUIREMENT ANALYSIS

Requirement analysis is an important phase in the development of the **Library Management Application** because it defines the expected behavior of the system and identifies the responsibilities of each user group. The application is designed to support both reader-oriented services and administrative library operations through a mobile environment.

The system must provide secure authentication so that users can access functions according to their assigned roles. After successful authentication, Readers can access functions related to books and their personal borrowing activities, while Administrators and Librarians can perform operational and management tasks.

The requirement analysis also focuses on maintaining accurate information about book availability, borrowing records, reservation requests, and user accounts. When a book is borrowed, returned, reserved, or renewed, the corresponding data must be updated consistently to reflect its latest state.

In addition, the system must provide an interface that is easy to understand and suitable for mobile devices. Common reader activities such as searching for books, viewing book information, checking borrowing deadlines, and making reservation requests should be completed through simple and intuitive interactions.

For administrative users, the application must provide efficient access to management functions so that daily library activities can be processed accurately and with minimal manual effort.

### 2.2.1 Major Features

The **Library Management Application** provides a set of major features designed to support both Readers and library administrators.

The first major feature is user authentication and profile management. Users can register an account, log in, log out, update personal information, and change their password. Personal information may include the user's name, phone number, and avatar. Authentication ensures that system functions are displayed according to the user's role.

Another major feature is book searching and discovery. Readers can search for books using keywords related to book titles or authors. Search results can also be filtered according to book categories and publication years. After selecting a book, users can view detailed information such as its summary, number of pages, publisher, available quantity, and storage location.

The book reservation feature allows Readers to submit a request to reserve a book. This function can be used when a book is temporarily unavailable or when a Reader wants the library to hold the book for later collection. Reservation requests can subsequently be reviewed and approved by authorized library staff.

Borrowing management is another important feature of the system. Readers can view the books they are currently borrowing together with borrowing dates and due dates. They can also submit renewal requests when they need additional borrowing time. A renewal should only be accepted when the book is eligible for renewal and is not currently reserved by another Reader.

The system also supports book inventory management for Administrators and Librarians. Authorized users can add, edit, or remove book information, update available quantities, classify books into categories, and define their physical storage locations.

Borrowing and returning operations are supported through administrative functions. Librarians can process borrowing transactions and confirm returned books. During the return process, the physical condition of a book can be recorded, such as normal, damaged, or lost.

The system further includes account management functions for Readers and Librarians. Administrative users can maintain user information and lock or unlock library cards when necessary.

System configuration is provided to allow Administrators to maintain core library regulations, including the maximum borrowing period, maximum number of books that may be borrowed, and overdue penalty rules.

Finally, the application provides statistical functions that allow Administrators to monitor current library activities. These functions include information about borrowed books, overdue books, frequently borrowed books, and newly registered Readers. Additional functions such as borrowing history, penalty management, digital library cards, and penalty revenue reports are considered optional features of the project.

### 2.2.2 Non-Functional Requirements

In addition to functional requirements, the **Library Management Application** must satisfy several non-functional requirements related to usability, security, reliability, performance, maintainability, and data integrity.

The application should provide a simple and intuitive mobile interface so that users with different levels of technical experience can use the system without difficulty. Navigation between major functions such as book searching, book details, borrowing information, reservations, and profile management should be clear and consistent.

Security is an important requirement because the system stores user accounts and personal information. Authentication must be required before users can access protected functions. Administrative operations must only be available to users who have the appropriate role and permissions. Password-related information should also be handled securely by the implementation.

The system should maintain reliable and consistent data. Changes to book quantities, borrowing status, reservations, and return records must be reflected accurately so that users do not receive incorrect information about book availability.

The application should also provide acceptable response times for common operations such as login, book searching, loading book details, viewing borrowing records, and updating user information. User actions should not experience unnecessary delays under normal usage conditions.

Maintainability is another important requirement. The system should be organized into logical modules so that functions related to authentication, books, borrowing, reservations, users, and system configuration can be modified or extended without significantly affecting unrelated components.

The application should also provide appropriate validation for user input. Invalid or incomplete data should be detected before being stored, while meaningful messages should be displayed to help users correct their input.

### 2.2.3 Functional Requirements

The functional requirements of the **Library Management Application** describe the specific actions that the system must perform in response to interactions from Readers, Administrators, and Librarians.

The system must allow users to register, log in, log out, update their personal information, and change their passwords. After authentication, the system must identify the user's role and provide the corresponding functionality.

For Readers, the application must support book searching, filtering, book detail viewing, reservation requests, borrowing record viewing, and renewal requests. Optional Reader functions include borrowing history, penalty information, and electronic library cards using QR codes or barcodes.

For administrative users, the application must support book management, category management, storage location management, borrowing and returning operations, reservation processing, Librarian account management, Reader management, system configuration, and statistical reporting.

#### *2.2.3.1 Actors*

The operational environment of the **Library Management Application** consists primarily of three actors: the **Reader**, the **Librarian**, and the **Administrator**.

The **Reader** represents the primary end-user of the application. Readers can register and authenticate their accounts, manage personal information, search for books, view book details, reserve books, monitor current borrowing records, and request borrowing extensions. Depending on the final implementation, Readers may also view borrowing history, penalty information, and an electronic library card.

The **Librarian** is responsible for daily library operations. Librarians can process borrowing and returning transactions, review reservation requests, inspect the physical condition of returned books, and access relevant Reader information required for library operations.

The **Administrator** possesses the highest level of management authority within the system. Administrators can manage books, categories, storage locations, Readers, Librarians, borrowing policies, penalty configurations, and statistical information. The Administrator may also perform operational functions available to Librarians when required.

### *2.2.3.2 Use Cases Diagram*

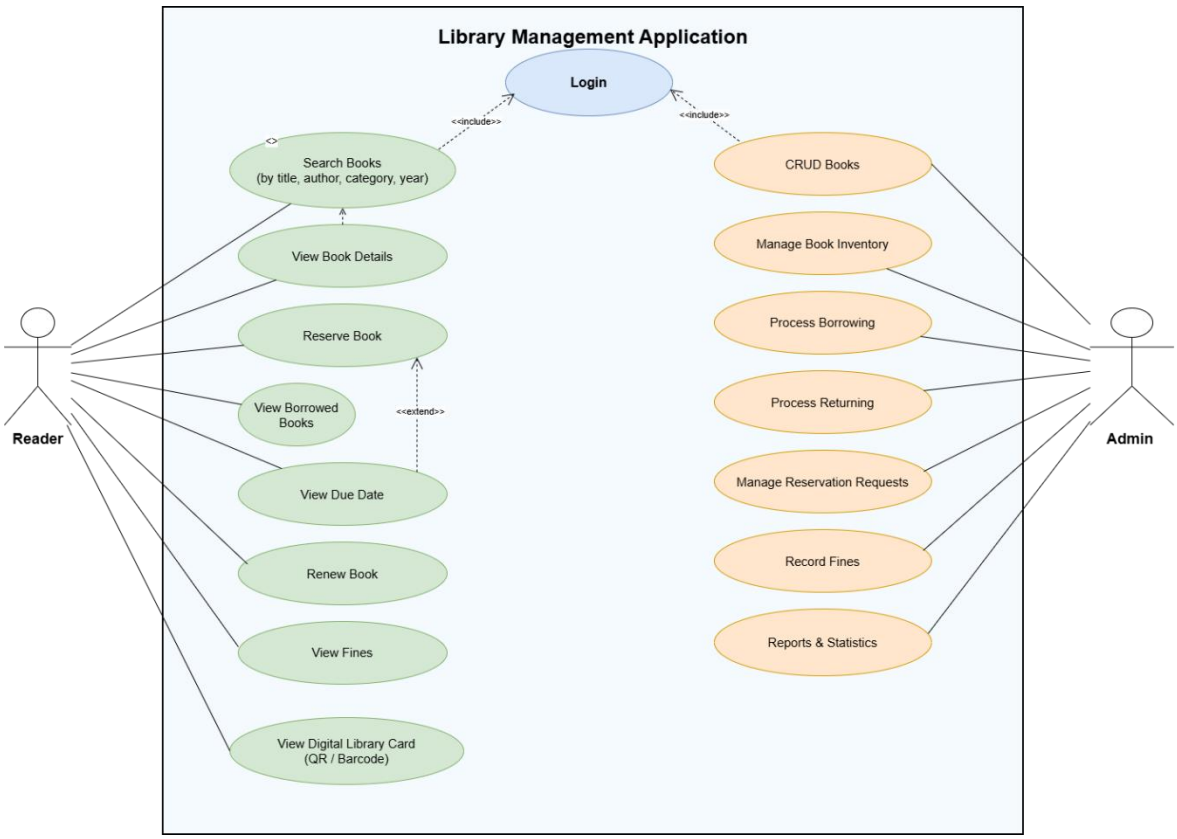



Image 1.General Use Case Diagram

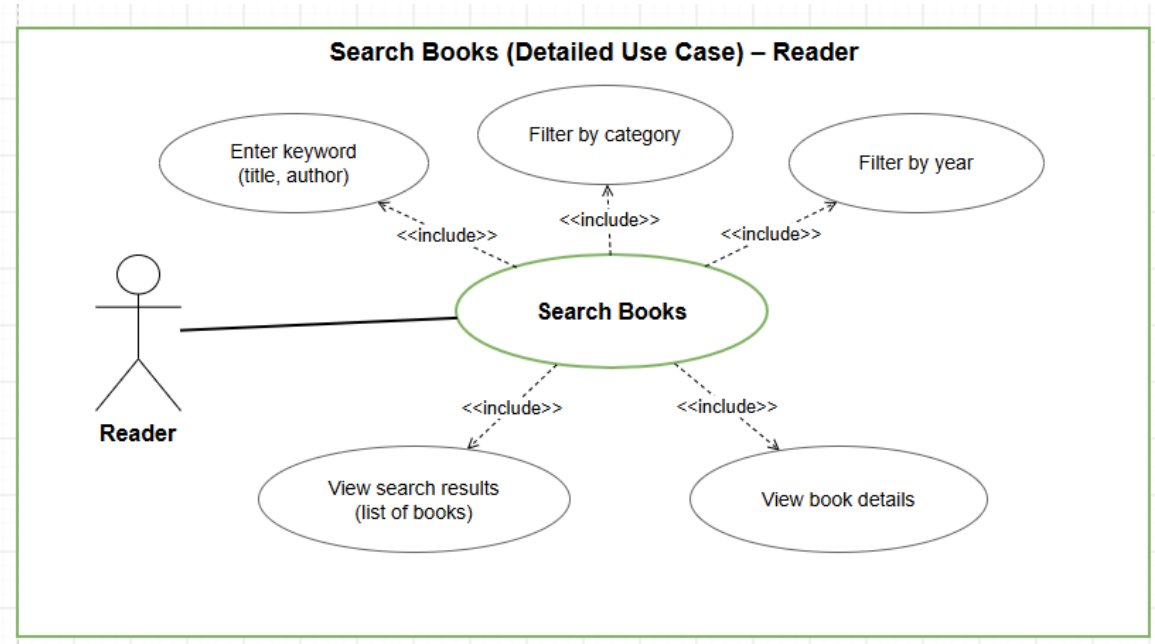



Image 2.Detailed Use Case Diagram: Search Book

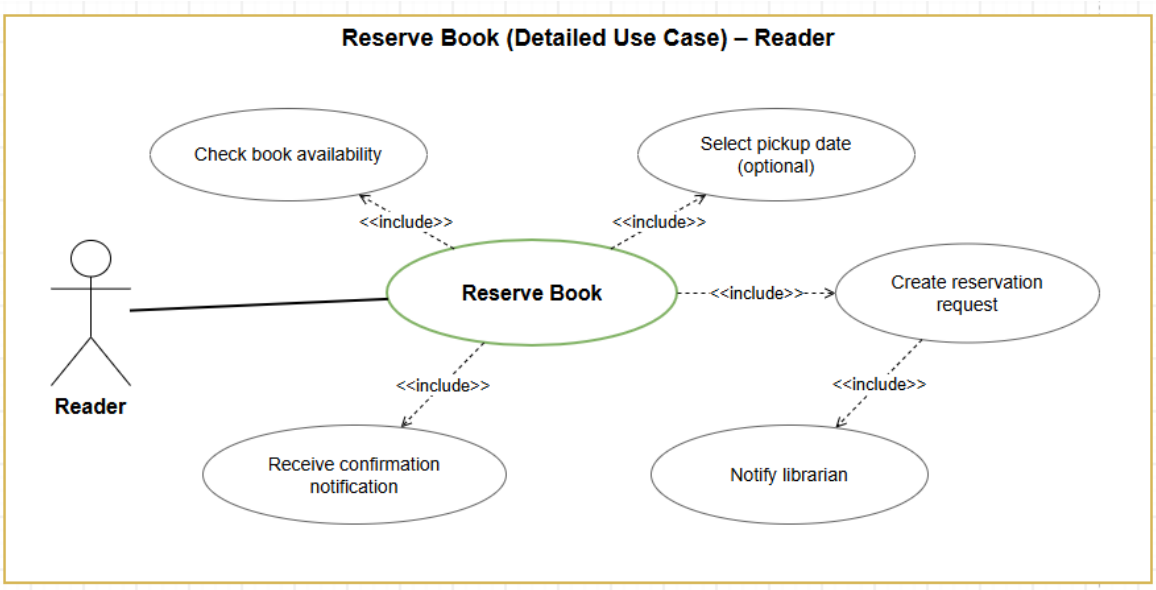



Image 3. Detailed Use Case Diagram: Reserve Book

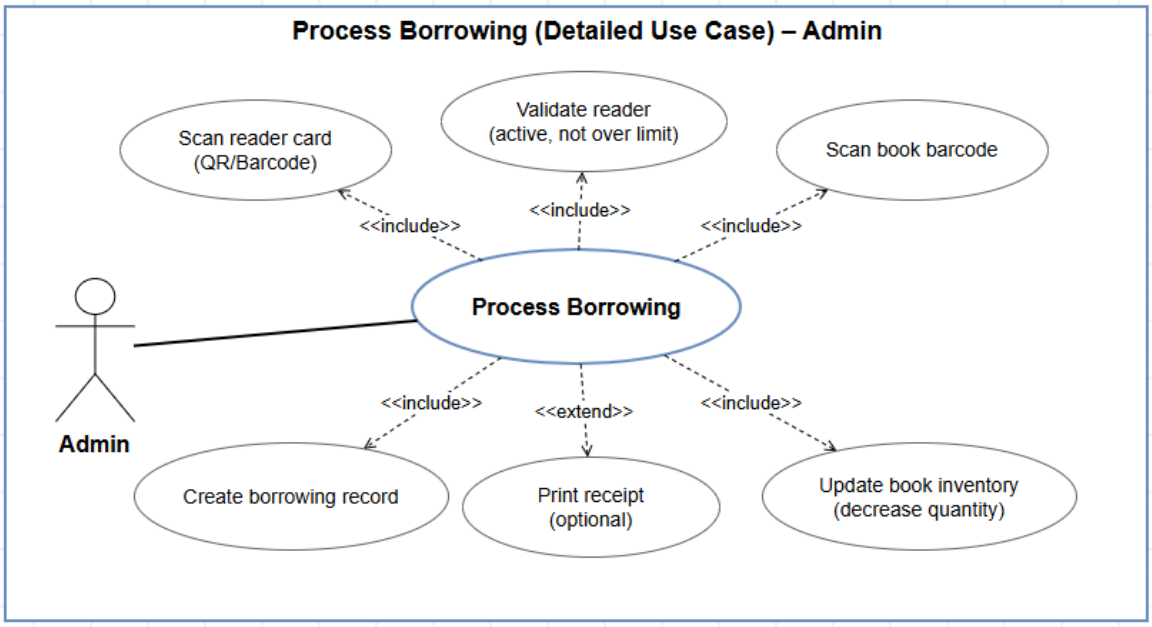



Image 4. Detailed Use Case Diagram: Process Borrowing

### *2.2.3.3 Use Cases Description*

Table 1. Login

| Field | Description |
|---|---|
| Use Case ID | UC-01 |
| Use Case Name | Login |
| Description | Allows users to authenticate and access the Library Management Application according to their role. |
| Actor | Reader, Admin |
| Pre-conditions | The user already has an active account in the system. |
| Flow of Events | **Basic Flow:** User opens the Login screen → enters account credentials → system validates the information → system grants access to the appropriate interface. |

Table 2. Search Books

| Field | Description |
|---|---|
| Use Case ID | UC-02 |
| Use Case Name | Search Books |

| | |
|---|---|
| Description | Allows Readers to search for books by title or author and filter books by category or publication year. |
| Actor | Reader |
| Pre-conditions | The application is accessible and book information exists in the system. |
| Flow of Events | **Basic Flow:** Reader opens the Search Books function → enters a keyword such as title or author → optionally selects category or publication year → system searches the database → matching books are displayed. |

Table 3. View Book Details

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-03 |
| Use Case Name | View Book Details |
| Description | Allows Readers to view detailed information about a selected book. |
| Actor | Reader |
| Pre-conditions | The selected book exists in the system. |
| Flow of Events | **Basic Flow:** Reader selects a book from the book list → system retrieves detailed book information → system displays the book summary, |

| | |
|---|---|
| | number of pages, publisher, available quantity, and shelf location. |

Table 4. Reserve Book

| Field | Description |
|---|---|
| Use Case ID | UC-04 |
| Use Case Name | Reserve Book |
| Description | Allows a Reader to reserve a book for later borrowing. |
| Actor | Reader |
| Pre-conditions | Reader is logged in and the selected book is available for reservation. |
| Flow of Events | **Basic Flow:** Reader selects a book → system checks book availability → Reader optionally selects a pickup date → Reader submits reservation → system creates a reservation request → Librarian/Admin is notified → Reader receives a confirmation notification. |

Table 5. View Borrowed Books

| Field | Description |
|---|---|
| Use Case ID | UC-05 |
| Use Case Name | View Borrowed Books |

| | |
|---|---|
| Description | Allows Readers to view the books they are currently borrowing. |
| Actor | Reader |
| Pre-conditions | Reader is logged in and has at least one active borrowing record. |
| Flow of Events | **Basic Flow:** Reader opens the borrowing management section → system retrieves active borrowing records → system displays the list of currently borrowed books. |

Table 6. View Due Date

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-06 |
| Use Case Name | View Due Date |
| Description | Allows Readers to view the expected return date of currently borrowed books. |
| Actor | Reader |
| Pre-conditions | Reader has an active borrowing record. |
| Flow of Events | **Basic Flow:** Reader selects a borrowed book → system retrieves the borrowing information → borrowing date and due date are displayed. |

Table 7. Renew Book

| Field | Description |
|---|---|
| Use Case ID | UC-07 |
| Use Case Name | Renew Book |
| Description | Allows a Reader to extend the borrowing period of a book when renewal conditions are satisfied. |
| Actor | Reader |
| Pre-conditions | Reader is logged in, the borrowing record is active, and the book is eligible for renewal. |
| Flow of Events | **Basic Flow:** Reader selects a borrowed book → selects Renew Book → system checks whether another Reader has reserved the book → if eligible, the system extends the due date → updated borrowing information is displayed. |

Table 8. View Fines

| Field | Description |
|---|---|
| Use Case ID | UC-08 |
| Use Case Name | View Fines |
| Description | Allows Readers to view penalty information related to overdue, damaged, or lost books. |
| Actor | Reader |
| Pre-conditions | Reader is logged in and penalty information exists. |

| | |
|---|---|
| Flow of Events | **Basic Flow:** Reader opens the Fine section → system retrieves penalty records → system displays penalty reason, amount, and payment status. |

Table 9. View Digital Library Card

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-09 |
| Use Case Name | View Digital Library Card |
| Description | Allows Readers to display their electronic library card using a QR code or barcode. |
| Actor | Reader |
| Pre-conditions | Reader is logged in and has an active library account. |
| Flow of Events | **Basic Flow:** Reader opens Digital Library Card → system retrieves Reader information → system generates and displays the QR code or barcode → Admin/Librarian can scan the code during borrowing operations. |

Table 10. CRUD Books

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-10 |
| Use Case Name | CRUD Books |

| | |
|---|---|
| Description | Allows Admin/Librarian to create, view, update, and delete book information. |
| Actor | Admin / Librarian |
| Pre-conditions | Admin/Librarian is logged in and has permission to manage books. |
| **Flow of Events** | **Basic Flow:** Admin opens book management → selects Add, Edit, or Delete → enters or modifies book information → system validates the data → changes are saved to the database. |

Table 11. Manage Book Inventory

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-11 |
| Use Case Name | Manage Book Inventory |
| Description | Allows Admin/Librarian to manage the quantity and availability of books in the library. |
| Actor | Admin / Librarian |
| Pre-conditions | Admin/Librarian is logged in and book records exist. |
| Flow of Events | **Basic Flow:** Admin selects a book → views current inventory → updates the total or available quantity → system validates and saves the updated inventory information. |

Table 12. Process Borrowing

| Field | Description |
|---|---|
| Use Case ID | UC-12 |
| Use Case Name | Process Borrowing |
| Description | Allows Admin/Librarian to create a borrowing transaction by scanning the Reader card and book barcode. |
| Actor | Admin / Librarian |
| Pre-conditions | Reader account is active, Reader has not exceeded the borrowing limit, and the selected book is available. |
| Flow of Events | **Basic Flow:** Admin scans Reader QR code/barcode → system validates Reader account and borrowing limit → Admin scans the book barcode → system checks book availability → system creates a borrowing record → available book quantity is decreased → borrowing process is completed. |
| Alternative Flow | If the Reader account is inactive, the borrowing limit is exceeded, or the book is unavailable, the system rejects the transaction and displays an appropriate message. |

Table 13. Process Returning

| Field | Description |
|---|---|
| Use Case ID | UC-13 |
| Use Case Name | Process Returning |

| | |
|---|---|
| Description | Allows Admin/Librarian to confirm returned books and update their physical condition. |
| Actor | Admin / Librarian |
| Pre-conditions | An active borrowing record exists for the returned book. |
| Flow of Events | **Basic Flow:** Admin identifies the borrowing record → confirms the returned book → checks book condition → records the condition as normal, damaged, or lost → system updates the borrowing status → book inventory is updated. |

Table 14. Manage Reservation Requests

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-14 |
| Use Case Name | Manage Reservation Requests |
| Description | Allows Admin/Librarian to review and process reservation requests submitted by Readers. |
| Actor | Admin / Librarian |
| Pre-conditions | Admin/Librarian is logged in and reservation requests exist. |
| Flow of Events | **Basic Flow:** Admin opens the reservation request list → selects a request → reviews Reader and book information → approves or rejects the request → |

| | |
|---|---|
| | system updates the reservation status → Reader is notified of the result. |

Table 15. Record Fines

| Field | Description |
|---|---|
| Use Case ID | UC-15 |
| Use Case Name | Record Fines |
| Description | Allows Admin/Librarian to record penalty charges for overdue, damaged, or lost books. |
| Actor | Admin / Librarian |
| Pre-conditions | A borrowing transaction exists and a penalty condition has been identified. |
| Flow of Events | **Basic Flow:** Admin selects the borrowing record → determines the penalty reason → system calculates or Admin enters the penalty amount → penalty record is created → penalty information is associated with the Reader account. |

Table 16. Reports & Statistics

| Field | Description |
|---|---|
| Use Case ID | UC-16 |
| Use Case Name | Reports & Statistics |
| Description | Allows Admin/Librarian to monitor and analyze library operational information. |

| | |
|---|---|
| Actor | Admin / Librarian |
| Pre-conditions | Relevant library transaction data exists in the system. |
| Flow of Events | **Basic Flow:** Admin opens Reports & Statistics → selects the required report → system retrieves and processes relevant data → results are displayed. Reports may include borrowed books, overdue books, most borrowed books, new Readers, and penalty revenue when implemented. |

# Chapter 3 . SYSTEM DESIGN

## 3.1 SYSTEM OVERVIEW

## 3.1 SYSTEM OVERVIEW

The **Library Management Application** is a mobile-based system designed to support the main activities of library management through Android devices. The application is developed using **Android Studio**, while **SQLite** is used as the local relational database for storing and managing application data.

The system is designed for two main user groups: **Reader** and **Admin/Librarian**. Each group is provided with different functions based on its responsibilities within the library. Readers mainly use the application to search for books, view detailed book information, reserve books, manage personal borrowing information, renew borrowed books, view fines, and access a digital library card.

Admin/Librarian users are responsible for managing books, processing borrowing and returning transactions, managing reservation requests, recording fines, maintaining user accounts, configuring library rules, and viewing reports and statistics.

The authentication module allows users to log in before accessing protected functions. After a successful login, the system identifies the user role and displays the corresponding interface. This role-based approach helps ensure that each user can only access the functions relevant to their responsibilities.

For Readers, the book searching module allows users to search books by title or author and filter results by category or publication year. After selecting a book, Readers can view information such as the book summary, number of pages, publisher, available quantity, and shelf location. If needed, the Reader can submit a reservation request for the selected book.

The personal borrowing management module allows Readers to view the books they are currently borrowing, together with borrowing dates and due dates. Readers can also request a renewal when the book is eligible and no other Reader has reserved it. Optional functions allow Readers to view borrowing history, penalty information, and a digital library card represented by a QR code or barcode.

For Admin/Librarian users, the book management module supports creating, updating, and deleting book information, as well as updating book quantities and classifications. The borrowing module allows staff to scan the Reader card and book barcode, validate borrowing conditions, create a borrowing record, and update book

availability. The returning module allows staff to confirm returned books and record their physical condition as normal, damaged, or lost.

The reservation management module allows Admin/Librarian users to review reservation requests submitted by Readers and approve or reject them. The system also provides functions for managing Reader and staff accounts, maintaining book categories and storage locations, configuring borrowing policies, and recording fines when necessary.

The reporting and statistical functions provide information about current library operations, including the number of borrowed books, overdue books, most frequently borrowed books, and newly registered Readers. Optional reporting can also include penalty revenue.

Overall, the **Library Management Application** integrates Reader services and library management operations into a centralized Android application. By using Android Studio for application development and SQLite for data storage, the system provides a structured solution suitable for managing the core functions of a library in an academic project environment.

## 3.2 DATABASE DESIGN

Database design is an important part of the **Library Management Application** because the system needs to store and manage different types of information, including users, books, borrowing transactions, reservations, fines, notifications, and system configurations. A well-structured database helps ensure that data is stored consistently and can be retrieved efficiently by the mobile application.

The system uses **SQLite** as its relational database. SQLite is suitable for this project because it can be integrated directly into an Android application without requiring a separate database server. This simplifies the development, testing, and deployment process in Android Studio.

The database is designed based on the main functional requirements of the system. User-related data is stored to support authentication, profile management, role identification, and account status control. Book-related data contains information such as title, author, category, publisher, publication year, number of pages, available quantity, and storage location.

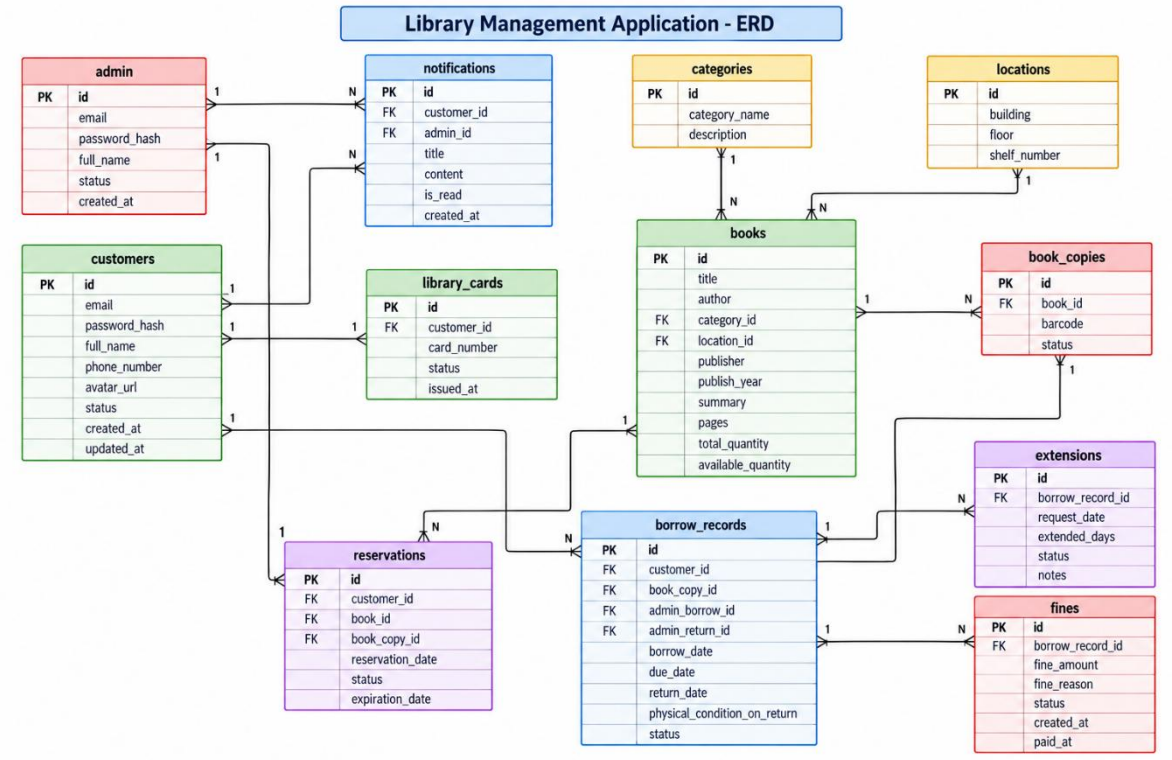
Borrowing information is stored separately so that the system can track which Reader has borrowed which book, the borrowing date, due date, return date, and current borrowing status. Reservation data is also maintained to support the book reservation feature, allowing Readers to request a book before borrowing it.

The database additionally stores fine information for overdue, damaged, or lost books. This information can be associated with the corresponding borrowing transaction and Reader account. System configuration data is used to maintain library regulations such as the maximum number of borrowing days, the maximum number of books that can be borrowed, and the fine rate per day.

Relationships between tables are established using primary keys and foreign keys. These relationships help maintain data integrity and allow the application to connect related information, such as linking a borrowing transaction to a specific Reader and Book, or associating a reservation request with a particular Reader and selected Book.

The database structure is designed to support the main operations of the application, including user authentication, book searching, book detail viewing, reservation processing, borrowing and returning management, fine recording, and report generation.

### 3.2.1 Entity Relationship Diagram (ERD)



Image(điền số). Entity Relationship Diagram

### 3.2.2 Database Tables Description

Table 17.Description Table

| Table Name | Description | Key Attributes |
|---|---|---|
| **admin** | Stores account information of administrators and librarians who manage library operations, including authentication information and account status. | - id (Primary Key)<br>- email<br>- password_hash<br>- full_name<br>- status<br>- created_at |
| **customers** | Stores account and personal information of Readers, including contact details, profile information, account status, and timestamps. | - id (Primary Key)<br>- email<br>- password_hash<br>- full_name<br>- phone_number<br>- avatar_url<br>- status |

| | | |
|---|---|---|
| | | - created_at<br><br>- updated_at |
| **notifications** | Stores notifications sent to Readers or Admin/Librarian users, including notification content and reading status. | - id (Primary Key)<br><br>- customer_id (Foreign Key → customers)<br><br>- admin_id (Foreign Key → admin)<br><br>- title<br><br>- content<br><br>- is_read<br><br>- created_at |
| **library_cards** | Stores electronic library card information associated with Readers and used for QR code or barcode identification. | - id (Primary Key)<br><br>- customer_id (Foreign Key → customers)<br><br>- card_number<br><br>- status<br><br>- issued_at |

| | | |
|---|---|---|
| **categories** | Stores book categories used to classify and organize books in the library system. | - id (Primary Key)<br>- category_name<br>- description |
| **locations** | Stores physical storage locations of books within the library, including building, floor, and shelf information. | - id (Primary Key)<br>- building<br>- floor<br>- shelf_number |
| **books** | Stores general bibliographic information of books, including title, author, category, location, publisher, publication year, quantity, and description. | - id (Primary Key)<br>- category_id (Foreign Key → categories)<br>- location_id (Foreign Key → locations)<br>- title<br>- author<br>- publisher<br>- publish_year<br>- summary |

| | | |
|---|---|---|
| | | - pages<br><br>- total_quantity<br><br>- available_quantity |
| **book_copies** | Stores information about each physical copy of a book. Each copy is identified by a barcode and has its own status. | - id (Primary Key)<br><br>- book_id (Foreign Key → books)<br><br>- barcode<br><br>- status |
| **reservations** | Stores reservation requests submitted by Readers for books or specific book copies, together with reservation status and expiration date. | - id (Primary Key)<br><br>- customer_id (Foreign Key → customers)<br><br>- book_id (Foreign Key → books)<br><br>- book_copy_id (Foreign Key → book_copies)<br><br>- reservation_date<br><br>- status |

| | | |
|---|---|---|
| | | - expiration_date |
| **borrow_records** | Stores borrowing and returning transactions, including Reader information, physical book copy, responsible Admin/Librarian, borrowing dates, return information, and physical condition of the returned book. | - id (Primary Key)<br>- customer_id (Foreign Key → customers)<br>- book_copy_id (Foreign Key → book_copies)<br>- admin_borrow_id (Foreign Key → admin)<br>- admin_return_id (Foreign Key → admin)<br>- borrow_date<br>- due_date<br>- return_date<br>- physical_condition_on_return<br>- status |

| | | |
|---|---|---|
| **extensions** | Stores borrowing extension requests associated with existing borrow records, including requested extension days and approval status. | - id (Primary Key)<br>- borrow_record_id (Foreign Key → borrow_records)<br>- request_date<br>- extended_days<br>- status<br>- notes |
| **fines** | Stores fines generated from overdue, damaged, or lost books and records their payment status. | - id (Primary Key)<br>- borrow_record_id (Foreign Key → borrow_records)<br>- fine_amount<br>- fine_reason<br>- status<br>- created_at<br>- paid_at |
| **Table Name** | **Description** | **Key Attributes** |

| | | |
|---|---|---|
| **admin** | Stores account information of administrators and librarians who manage library operations, including authentication information and account status. | - id (Primary Key)<br>- email<br>- password_hash<br>- full_name<br>- status<br>- created_at |
| **customers** | Stores account and personal information of Readers, including contact details, profile information, account status, and timestamps. | - id (Primary Key)<br>- email<br>- password_hash<br>- full_name<br>- phone_number<br>- avatar_url<br>- status<br>- created_at<br>- updated_at |
| **notifications** | Stores notifications sent to Readers or Admin/Librarian users, | - id (Primary Key) |

| | | |
|---|---|---|
| | including notification content and reading status. | - customer_id (Foreign Key → customers)<br>- admin_id (Foreign Key → admin)<br>- title<br>- content<br>- is_read<br>- created_at |
| **library_cards** | Stores electronic library card information associated with Readers and used for QR code or barcode identification. | - id (Primary Key)<br>- customer_id (Foreign Key → customers)<br>- card_number<br>- status<br>- issued_at |
| **categories** | Stores book categories used to classify and organize books in the library system. | - id (Primary Key)<br>- category_name<br>- description |

### 3.2.3 Relationships Between Tables

### 3.2.4 3.2.3 Relationships Between Tables

The database of the **Library Management Application** is designed using relational relationships between the main entities in order to maintain data consistency and support the system's borrowing, reservation, notification, and fine management processes.

The **admin** table has a one-to-many relationship with the **notifications** table. One Admin/Librarian account can be associated with multiple notification records, while each notification may reference one administrator through the admin_id foreign key. The **customers** table also has a one-to-many relationship with the **notifications** table through customer_id, allowing a Reader to receive multiple notifications from the system.

The **customers** table has a one-to-one relationship with the **library_cards** table. Each Reader is associated with one library card, while each library card belongs to one specific Reader. This relationship supports the electronic library card function, where the card number can be represented as a QR code or barcode.

The **categories** table has a one-to-many relationship with the **books** table. One category can contain multiple books, while each book belongs to one category through the category_id foreign key. Similarly, the **locations** table has a one-to-many relationship with the **books** table. One storage location may contain multiple books, while each book is assigned to one location through location_id.

The **books** table has a one-to-many relationship with the **book_copies** table. One book title can have multiple physical copies, while each physical copy belongs to one specific book through book_id. This structure allows the system to manage both general book information and individual physical copies with unique barcodes and statuses.

The **customers** table has a one-to-many relationship with the **reservations** table. A Reader can create multiple reservation requests over time, while each reservation belongs to one Reader through customer_id. The **books** table also has a one-to-many relationship with the **reservations** table through book_id. In addition, a reservation may be associated with a specific physical copy through book_copy_id.

The **customers** table has a one-to-many relationship with the **borrow_records** table. A Reader can have multiple borrowing transactions, while each borrowing record belongs to one Reader through customer_id. Each borrowing record is also associated with one specific physical book copy through book_copy_id.

The **admin** table is connected to the **borrow_records** table through both admin_borrow_id and admin_return_id. These relationships record which Admin/Librarian processed the borrowing transaction and which Admin/Librarian processed the return transaction. One Admin/Librarian can therefore process many borrowing and returning records.

The **borrow_records** table has a one-to-many relationship with the **extensions** table. One borrowing transaction may have one or more extension requests over time, while each extension request belongs to one borrowing record through borrow_record_id.

The **borrow_records** table also has a one-to-many relationship with the **fines** table. A borrowing transaction may generate one or more fine records when the book is returned late, damaged, or lost. Each fine is linked to the corresponding borrowing transaction through borrow_record_id.

Through these relationships, the database connects Readers, Admin/Librarian users, books, physical book copies, reservations, borrowing transactions, extension requests, fines, and notifications in a consistent structure. This design provides the data foundation required for the main functions of the Library Management Application.

## 3.3 UI/UX DESIGN (CHỪA)

## 3.4 CLASS DIAGRAM

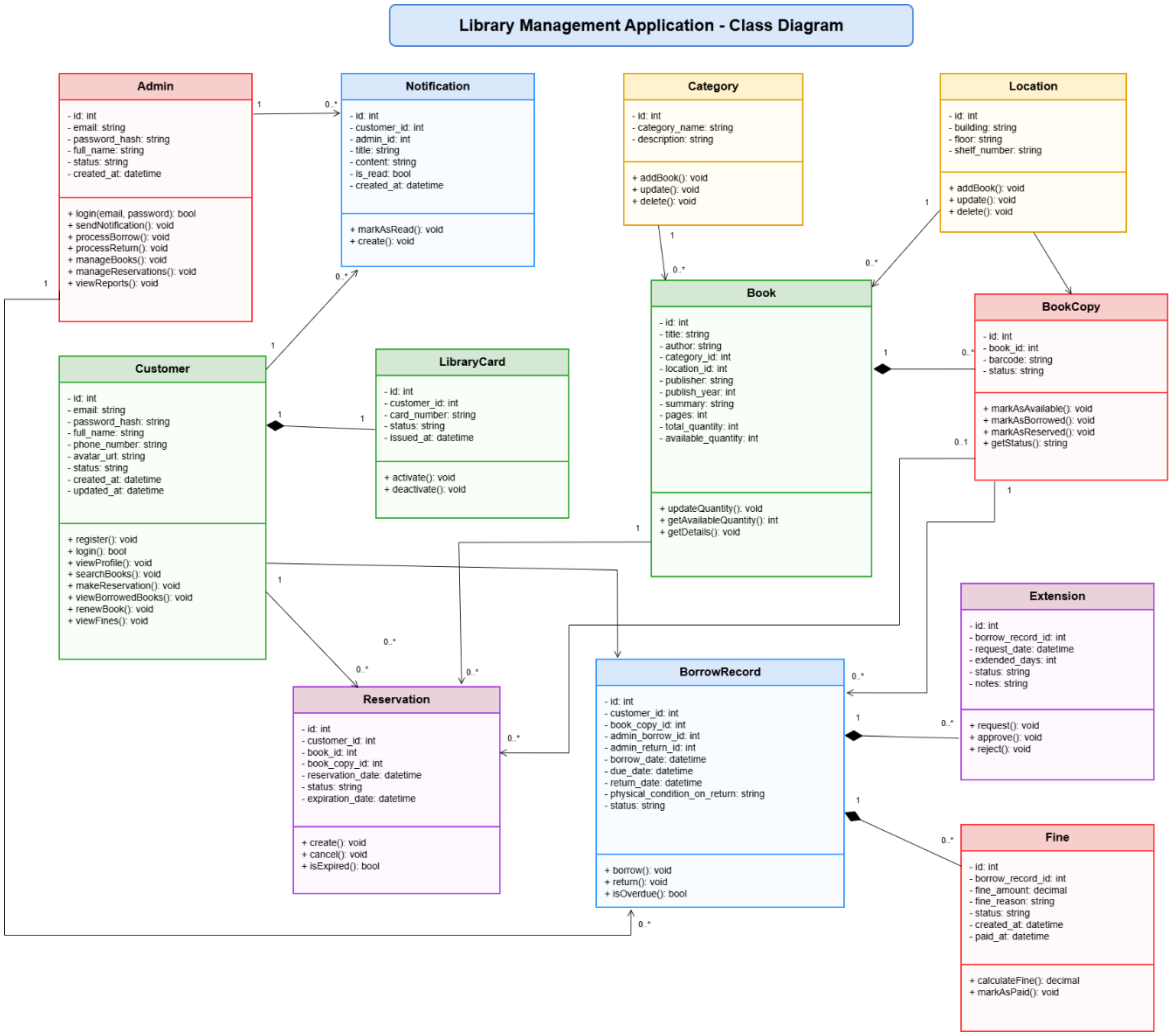



Image 3.Class Diagram

### 3.4.1 Class Diagram Description

The Class Diagram of the Library Management Application illustrates the structural organization of the main entities and their responsibilities within the system. The diagram includes classes related to user management, book management, borrowing operations, reservations, library cards, notifications, extensions, and fines.

The Admin class represents administrators and librarians who manage library operations. It stores account information such as id, email, password_hash, full_name, status, and created_at. The class provides operations for authentication, sending notifications, processing borrowing and returning transactions, managing books and reservations, and viewing reports.

The Customer class represents Readers of the library. It contains personal and account information including id, email, password_hash, full_name, phone_number, avatar_url, status, created_at, and updated_at. Its main operations include registration, login, profile management, book searching, creating reservations, viewing borrowed books, renewing books, and viewing fines.

The Notification class stores messages sent to users. It contains identifiers for both Customer and Admin accounts, together with title, content, is_read, and created_at. The class provides operations for creating notifications and marking them as read.

The LibraryCard class represents the electronic library card assigned to a Reader. It stores the card identifier, associated Customer, card number, status, and issue date. The class supports activating and deactivating the library card.

The Category class represents book categories used to classify books in the system. It stores id, category_name, and description. The class provides operations to add, update, and delete category information.

The Location class stores the physical storage location of books within the library, including the building, floor, and shelf number. It provides operations for adding, updating, and deleting location information.

The Book class represents general information about a book title. It contains attributes such as id, title, author, category_id, location_id, publisher, publish_year, summary, pages, total_quantity, and available_quantity. The class provides operations for updating book quantities, retrieving available quantities, and displaying detailed book information.

The BookCopy class represents an individual physical copy of a book. Each copy is associated with a Book through book_id and has its own barcode and status. The class provides operations for marking a copy as available, borrowed, reserved, and for retrieving its current status.

The Reservation class represents a book reservation created by a Reader. It stores references to the Customer, Book, and BookCopy together with reservation date, status, and expiration date. The class provides operations for creating and canceling reservations and checking whether a reservation has expired.

The BorrowRecord class represents a borrowing transaction. It stores references to the Customer, BookCopy, the Admin who processed the borrowing, and the Admin who processed the return. It also contains the borrowing date, due date, return date, physical condition on return, and transaction status. The class provides operations for borrowing, returning, and checking whether a borrowing record is overdue.

The Extension class represents a renewal request associated with an existing borrowing record. It stores the borrowing record identifier, request date, requested

number of extended days, status, and notes. Its operations allow an extension request to be created, approved, or rejected.

The Fine class represents penalties generated from borrowing violations. It stores the associated borrowing record, fine amount, reason, status, creation date, and payment date. The class provides operations for calculating a fine and marking the fine as paid.

Overall, the Class Diagram separates responsibilities among the main classes while preserving the relationships required by the system. Customer-related classes support Reader activities, Admin-related operations support library management, and transactional classes such as Reservation, BorrowRecord, Extension, and Fine handle the core borrowing lifecycle of the Library Management Application.

## 3.5 ACTIVITY DIAGRAM

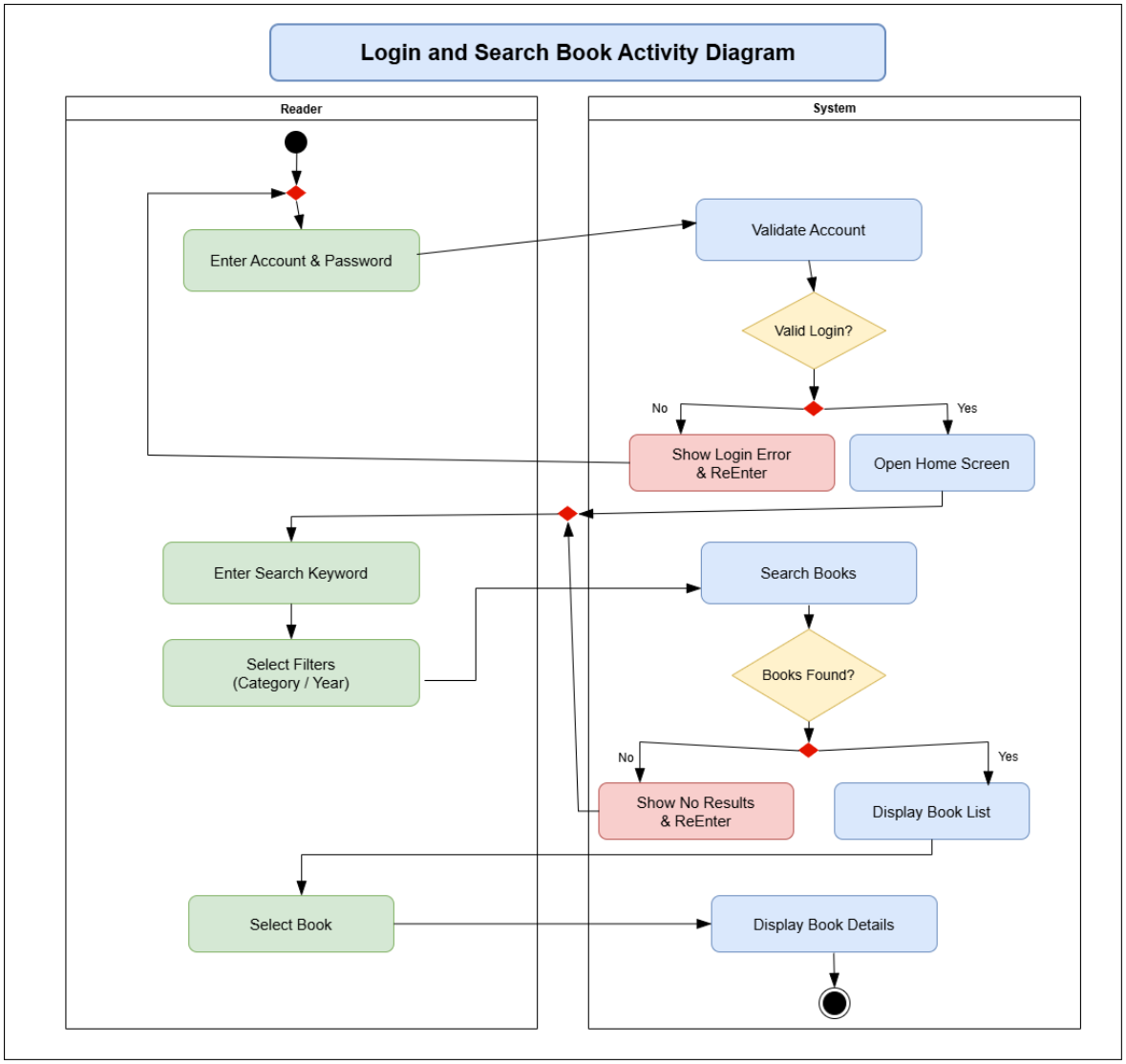



Image 4.Activity Diagram – Login and Search Book

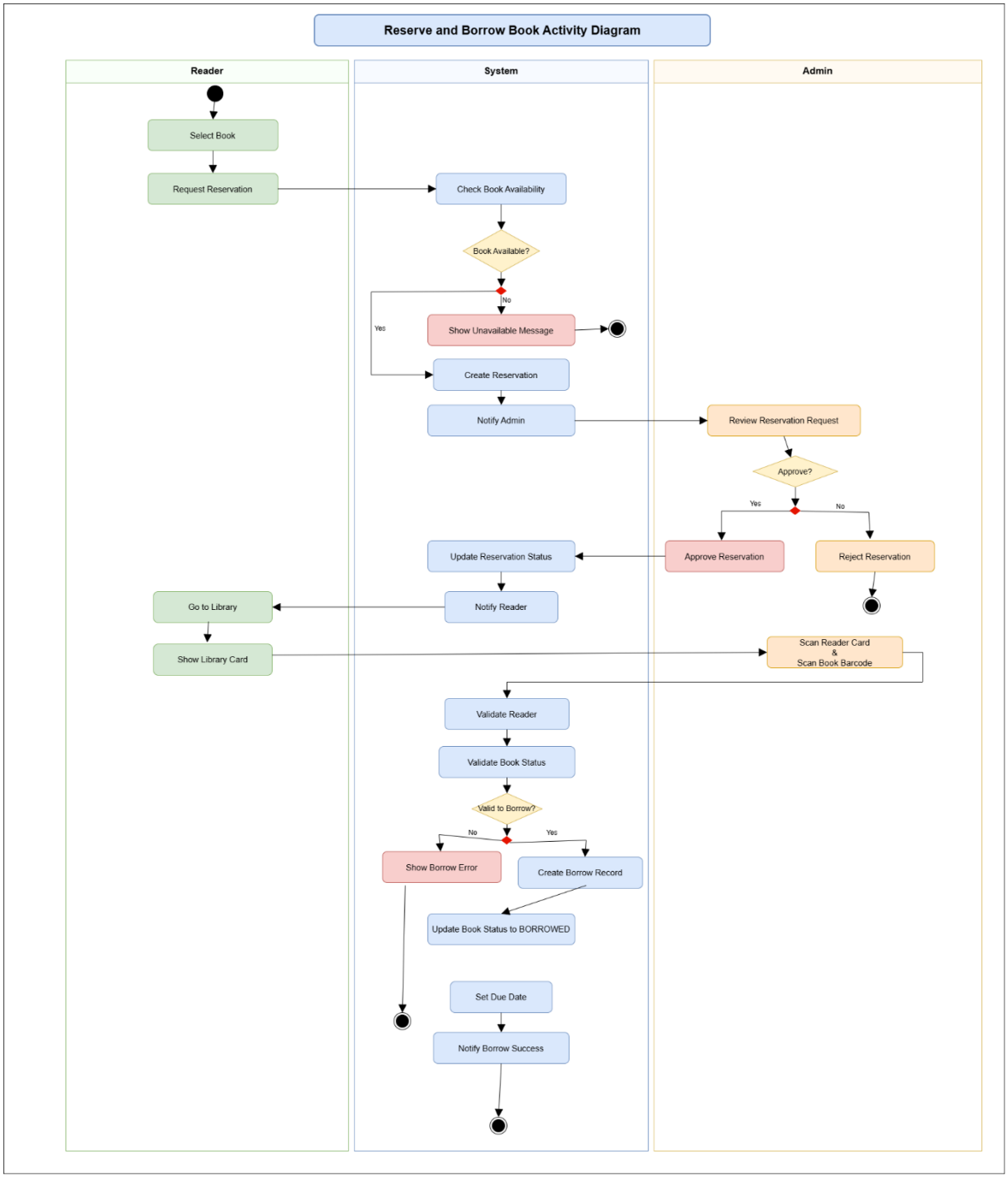



Image 5.Activity Diagram – Reserve and Borrow Book

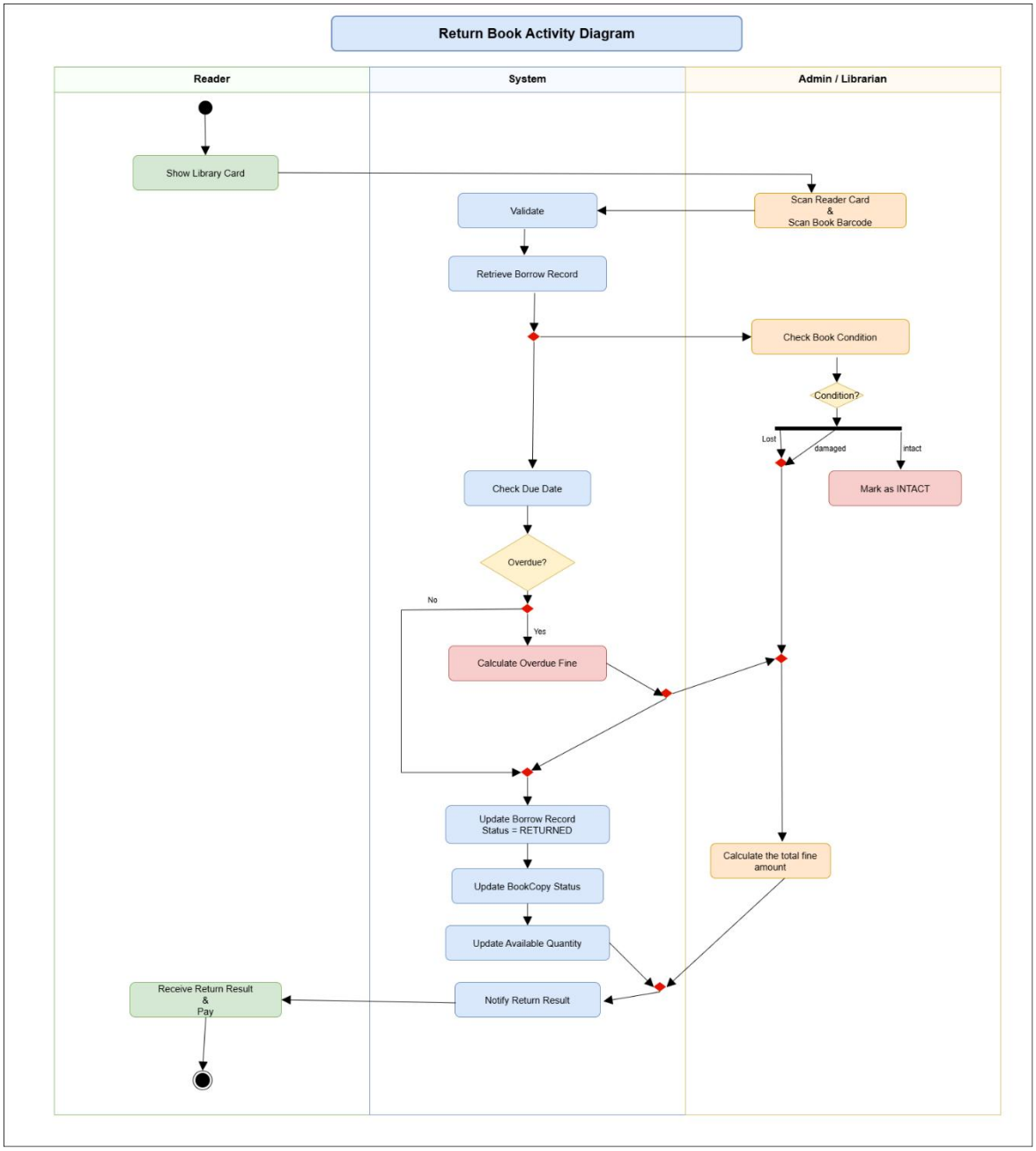



Image 6.Activity Diagram – Return Book

## 3.6 SEQUENCE DIAGRAM

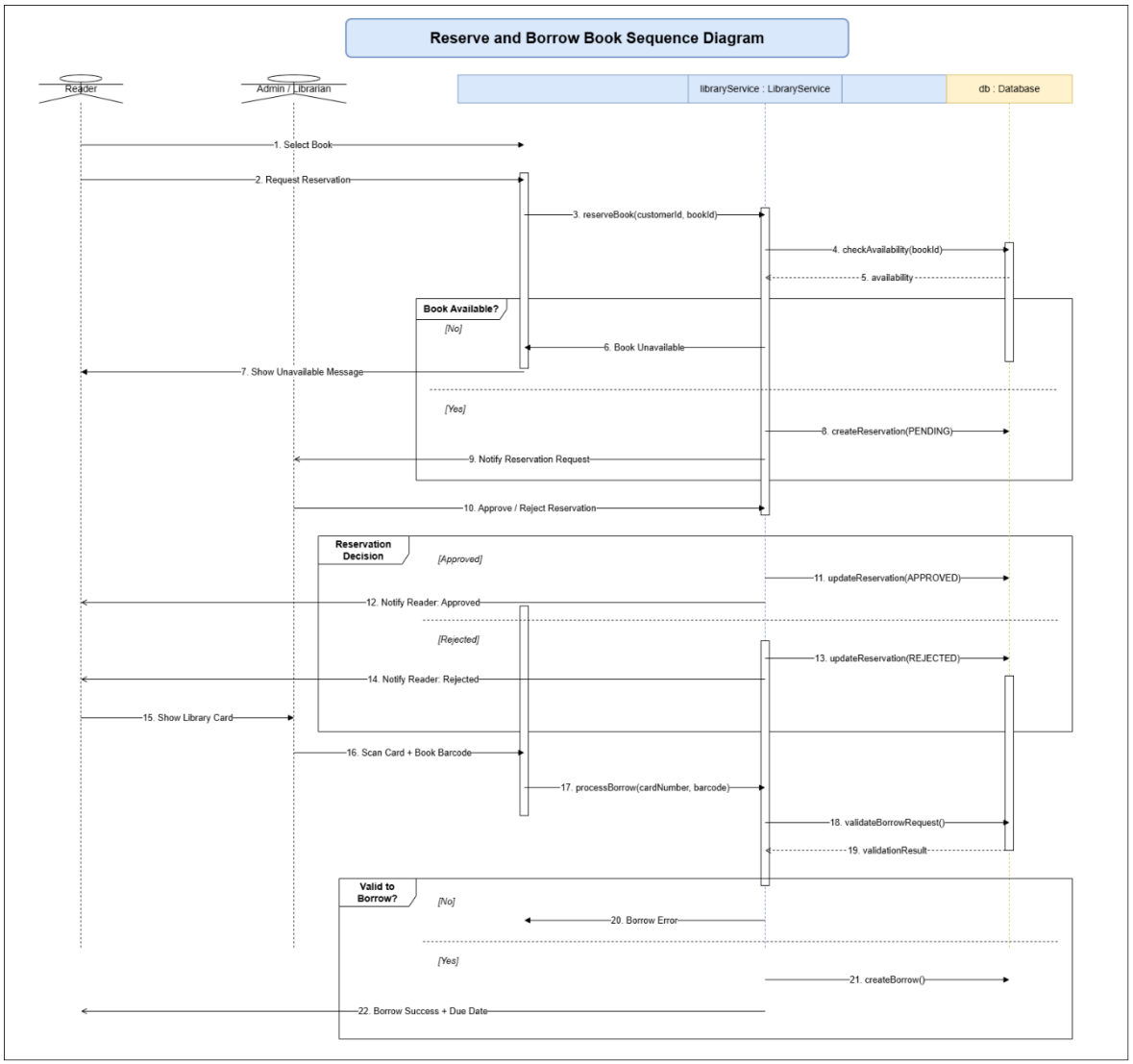



Image 7.Sequence Diagram - Reserve and Borrow Book

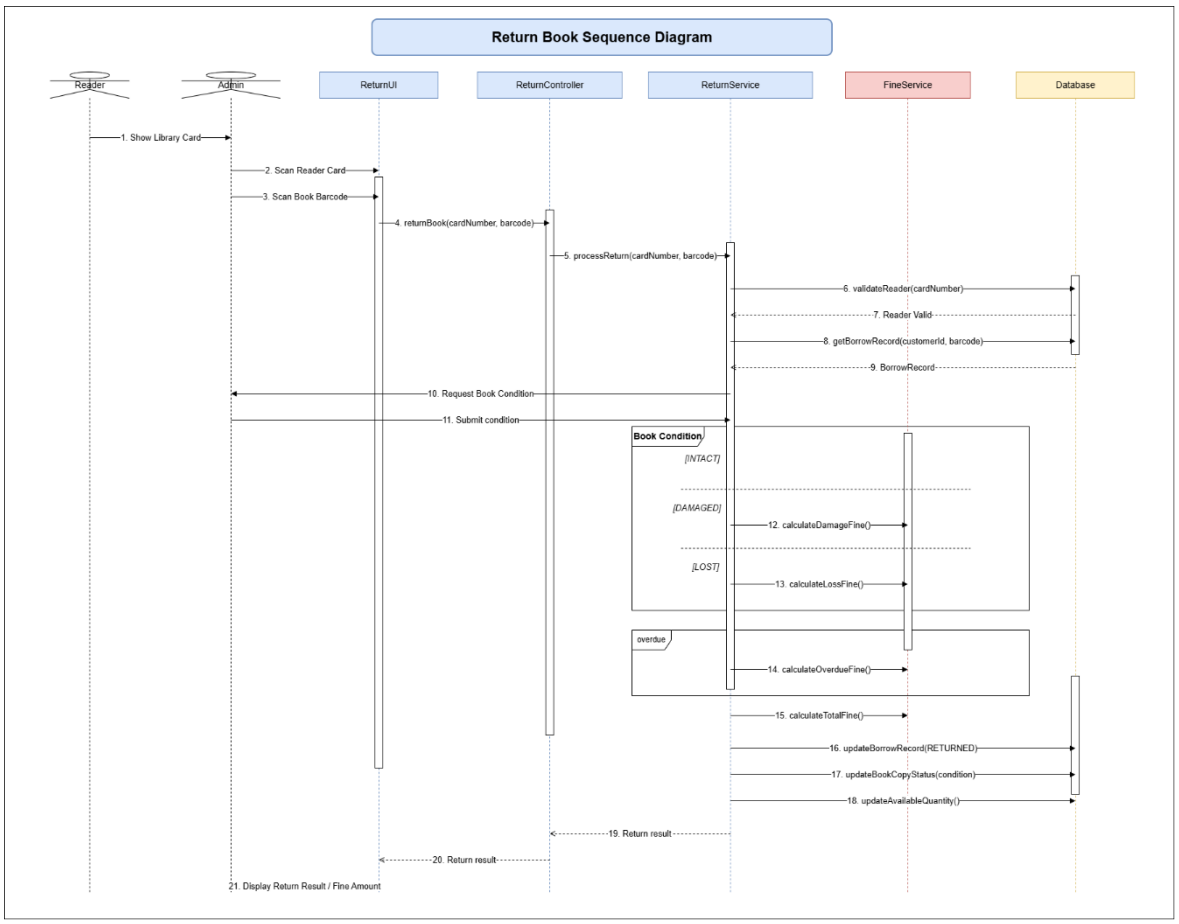



Image 8.Sequence Diagram – Return Book

## 3.7 DATA FLOW DIAGRAM (CHỪA, TỪ TỪ TÍNH)

## 3.8 SYSTEM ARCHITECTURE

The **Library Management Application** is designed using a layered architecture to separate the user interface, application logic, data access operations, and database management. This architecture helps organize the Android application into independent components, making the system easier to understand, maintain, test, and extend.

The system is developed using **Android Studio**, while **SQLite** is used as the local relational database. User interactions are handled through Android screens and interface components. These components communicate with the business logic layer, where the main

library rules and processing operations are implemented. The business logic layer then accesses data through the data access layer, which performs database operations on SQLite.

The architecture consists of four main layers: the **Presentation Layer**, **Business Logic Layer**, **Data Access Layer**, and **Database Layer**. Each layer has a specific responsibility and communicates with the adjacent layer instead of directly accessing unrelated components. This separation reduces code dependency and supports a more structured application design.

The Presentation Layer is responsible for displaying information and receiving input from Readers and Admin/Librarian users. The Business Logic Layer processes application rules such as book availability checking, borrowing validation, reservation processing, renewal eligibility, and fine calculation. The Data Access Layer handles data retrieval and modification, while the Database Layer stores all persistent information required by the application.

This architecture is suitable for the current academic project because it provides a clear separation of responsibilities while remaining simple enough to implement within an Android Studio and SQLite environment.

### 3.8.1 Presentation Layer

The **Presentation Layer** represents the user interface of the Library Management Application and is responsible for direct interaction with users. It includes Android screens used by Readers and Admin/Librarian users to access system functions.

For Readers, this layer provides interfaces for login, book searching, book details, reservations, borrowed book management, renewal requests, fines, and the digital library card. For Admin/Librarian users, it provides interfaces for book management, borrowing and returning processing, reservation management, Reader management, fine recording, and reports.

The Presentation Layer receives user input and displays system results. It does not directly access the SQLite database. Instead, user requests are passed to the Business Logic Layer for processing.

### 3.8.1 3.8.2 Business Logic Layer

The **Business Logic Layer** contains the main processing rules of the application. It receives requests from the Presentation Layer, validates the required conditions, and determines how the system should respond.

This layer handles operations such as user authentication, book searching, reservation validation, borrowing eligibility, returning processes, book renewal, inventory updates, and fine calculation.

For example, before a borrowing transaction is created, the system verifies that the Reader account is active, the borrowing limit has not been exceeded, and the selected book copy is available. Similarly, before a renewal request is approved, the system checks whether the book has already been reserved by another Reader.

By keeping these rules in a separate layer, the system can apply the same business rules consistently across different application screens.

### 3.8.2 Bussiness Logic Layer

The **Business Logic Layer** contains the main processing rules of the application. It receives requests from the Presentation Layer, validates the required conditions, and determines how the system should respond.

This layer handles operations such as user authentication, book searching, reservation validation, borrowing eligibility, returning processes, book renewal, inventory updates, and fine calculation.

For example, before a borrowing transaction is created, the system verifies that the Reader account is active, the borrowing limit has not been exceeded, and the selected book copy is available. Similarly, before a renewal request is approved, the system checks whether the book has already been reserved by another Reader.

By keeping these rules in a separate layer, the system can apply the same business rules consistently across different application screens.

### 3.8.1 Data Access Layer

The **Data Access Layer** is responsible for communication between the Business Logic Layer and the SQLite database. It performs the required Create, Read, Update, and Delete operations for application data.

This layer handles database operations related to Admin accounts, Customers, Books, Book Copies, Categories, Locations, Reservations, Borrow Records, Extensions, Fines, Notifications, and Library Cards.

The Data Access Layer receives data requests from the Business Logic Layer, executes the corresponding SQLite operations, and returns the results. This design prevents interface components from directly executing database queries and improves the maintainability of the application.

### 3.8.2 Database Layer

The Database Layer represents the persistent data storage of the Library Management Application. The system uses SQLite to store and manage relational data directly on the Android device.

The database contains tables including admin, customers, notifications, library_cards, categories, locations, books, book_copies, reservations, borrow_records, extensions, and fines.

Primary keys and foreign keys are used to maintain relationships between these tables. For example, a borrowing record connects a Customer with a specific BookCopy, while a Fine or Extension is linked to the corresponding BorrowRecord.

SQLite provides lightweight and persistent local storage without requiring a separate database server, making it appropriate for the implementation scope of the Library Management Application.

## 3.9 FOLDER STRUCTURE (CHỪA)

# Chapter 4 . IMPLEMENTATION & RESULTS

## 4.1 DEVELOPMENT ENVIRONMENT & TOOLS

The **Library Management Application** is developed as an Android mobile application using **Android Studio** as the primary Integrated Development Environment. Android Studio provides the necessary tools for designing application interfaces, managing source code, building the project, testing application functions, and running the application on Android devices or emulators.

The application uses **SQLite** as its local relational database management system. SQLite is integrated directly into the Android environment and does not require a separate database server. It is used to store and manage the application data, including Admin accounts, Reader accounts, books, book copies, categories, storage locations, library cards, reservations, borrowing records, extension requests, fines, and notifications.

The user interface of the application is developed using Android layout resources. These layouts define the structure of screens such as Login, Search Books, Book Details, Reservation, Borrowed Books, Digital Library Card, Book Management, Borrowing Processing, Returning Processing, and Reports.

Android Emulator or a physical Android device can be used during development to test the application. These testing environments allow the development team to verify interface behavior, navigation, database operations, and the main Reader and Admin/Librarian workflows.

The project is organized into different components according to their responsibilities. User interface components handle screen display and user interactions, business components process library rules and application logic, while database-related components communicate with SQLite to perform data storage and retrieval operations.

The selected development environment provides the necessary foundation for implementing the core functions of the Library Management Application in a mobile environment while keeping the project suitable for academic development and testing.

## 4.2 SYSTEM IMPLEMENTATION

The implementation of the **Library Management Application** is divided into functional modules based on the requirements and system design defined in the previous chapters.

The major implementation areas include authentication, book searching, book management, reservations, borrowing and returning operations, renewals, fines, electronic library cards, notifications, and reporting functions.

The authentication module handles access to the application. Users provide their account information through the Login screen, and the system checks the corresponding data stored in SQLite. After successful authentication, the system identifies whether the account belongs to a Reader or an Admin/Librarian and displays the appropriate functions.

For Readers, the book searching module allows users to search books by title or author and apply filters based on category or publication year. Search results are retrieved from the book-related database tables and displayed as a list. Selecting a book opens the Book Details screen, where information such as the title, author, publisher, publication year, summary, number of pages, available quantity, and shelf location can be viewed.

The reservation module allows a Reader to submit a reservation request for a selected book. The system checks the relevant book information and creates a record in the reservation data when the request satisfies the required conditions. Admin/Librarian users can later review and process these reservation requests.

The borrowing module is designed for Admin/Librarian users. During the borrowing process, the system identifies the Reader and the physical book copy, validates borrowing conditions, creates a borrowing record, and updates the status of the corresponding book copy. The borrowing date and due date are stored so that the application can track the Reader's active borrowing transactions.

The returning module allows Admin/Librarian users to identify an existing borrowing record and confirm that the book has been returned. The system records the return date and physical condition of the returned book. The status of the book copy is then updated so that it can become available again when appropriate.

The renewal module allows Readers to request additional borrowing time for an active borrowing record. The system checks the related borrowing and reservation information before the extension request can be processed. Approved extension requests update the borrowing period according to the permitted number of additional days.

Fine management is used when a borrowing transaction violates library regulations, such as late returns or books that are damaged or lost. The corresponding Fine record stores the amount, reason, status, creation time, and payment information.

The electronic library card function provides Readers with a digital representation of their library card. The card number can be displayed as a QR code or barcode and used by library staff during borrowing operations.

The reporting and statistics module provides Admin/Librarian users with summarized information related to library activities. The system can display statistics such as the number of borrowed books, overdue books, frequently borrowed books, and newly registered Readers. Penalty revenue statistics can also be included when the optional fine reporting function is implemented.

Overall, the implementation connects the Android user interfaces, application logic, and SQLite database to support the complete workflow of the Library Management Application.